

# Using ModelZoo models on your own datasets

Print to PDF

## Contents

- `mouse_pupil_vc` model
- Pigeon Pupil

Animal behavior has to be analyzed with painstaking accuracy. Therefore, animal pose estimation has been an important tool to study animal behavior precisely.

Beside providing an open source toolbox for researchers to develop customized deep neural networks for markerless pose estimation, we at DeepLabCut also aim to build robust, generalizable models. Part of this effort is via the [DeeplabCut ModelZoo](#).

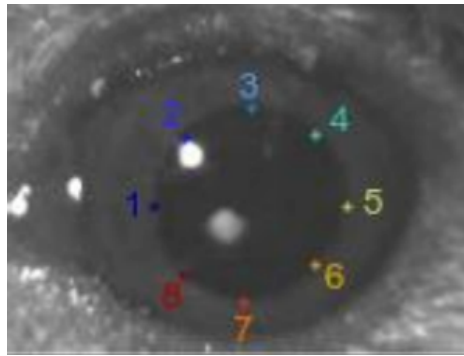
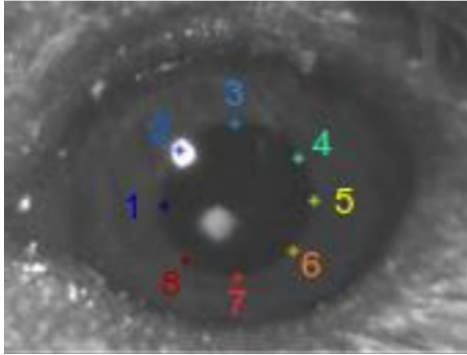
The Zoo hosts user-contributed and DLC-team developed models that are trained on specific animals and scenarios. You can analyze your videos directly with these models without training. The models have strong zero-shot performance on unseen out-of-domain data which can be further improved via pseudo-labeling. Please check the first [ModelZoo manuscript](#) for further details.

This recipe aims to show a usecase of the `mouse_pupil_vc` and is contributed by 2022 DLC AI Resident [Neslihan Wittek](#) .

## `mouse_pupil_vc` model

This model was contributed by Jim McBurney-Lin at University of California Riverside, USA. The model was trained on images

of C57/B6J mice eyes, and also then augmented with mouse eye data from the Mathis Lab at EPFL.



Since we would like to evaluate the models performance on out-of-domain data, we will analyze pigeon pupils. For more discussions and work on so-called out-of-domain data, see [Mathis, Biasi 2020](#).

## Pigeon Pupil

The eye pupil admits and regulates the amount of light entering the retina in order to enable image perception. Beside this crucial role, the pupil also reflects the state of the brain. The systemic behavior of the pupil has not been vastly studied in birds, although researchers from [Max Planck Institute for Ornithology in Seewiesen](#) have shed light on pupil behaviors in pigeons.

The pupils of male pigeons get smaller during courtship behavior. This is in contrast to mammals, for which the pupil size dilates in response to an increase in arousal. In addition, the pupil size of pigeons dilates during non-REM sleep, while they rapidly constrict during REM sleep. Examining these differences and the reason behind them, might be helpful to understand the pupillary behavior in general.

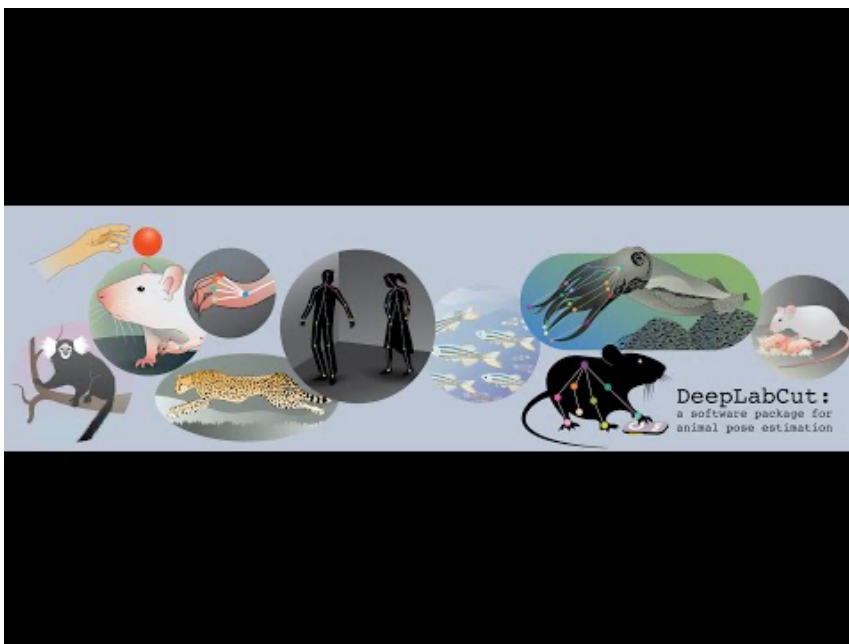
In light of these findings, we wanted to show whether the **mouse\_pupil\_vclose** model give us an accurate tracking performance for the pigeon pupil as well.

## Jupyter & Google Colab Notebook

DeepLabCut provides a Google Colab Notebook to analyze your video with a pretrained networks from the ModelZoo. No need for local installation of DeepLabCut!

Since we are interested in the accuracy of the **mouse\_pupil\_vclose** on pigeon pupil data, we will use a video which consists of 7 recordings of pigeon pupils.

Check the [ModelZoo Colab page](#) and a video tutorial on how to use the ModelZoo on Google Colab.



### Hint

You are happy with the model and want to go on analyzing further videos on your local machine or you want to refine the model for your specific usecase?

```
!zip -r /content/file.zip /content/pigeon_model
from google.colab import files
files.download("/content/file.zip")
```

## Analyze Videos at Your Local Machine

DeepLabCut host models from the [DeepLabCut ModelZoo Project](#).

The `create_pretrained_project` function will create a new project directory with the necessary sub-directories and a basic configuration file. It will also initialize your project with a pre-trained model from the DeepLabCut ModelZoo.

The rest of the code should be run within your DeepLabCut environment. Check [here](#) for the instructions for the DeepLabCut installation.

To initialize a new project directory with a pre-trained model

from the DeepLabCut ModelZoo, run the code below.

 **Warning**

This method is currently implemented for Tensorflow only, Pytorch compatibility is coming soon.

```
import deeplabcut

deeplabcut.create_pretrained_project(
    "projectname",
    "experimenter",
    [r"path_for_the_videos"],
    model="mouse_pupil_vclose",
    working_directory=r"project_directory",
    copy_videos=True,
    videotype=".mp4 or .avi?",
    analyzevideo=True,
    filtered=True,
    createlabeledvideo=True,
    trainFraction=None,
    engine=deeplabcut.Engine.TF,
)
```

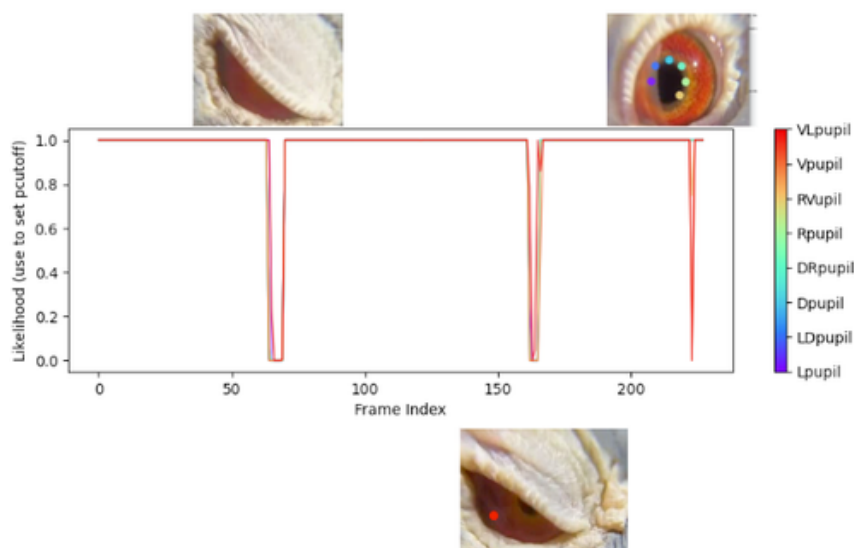
 **Important**

Your videos should be cropped around the eye for better model accuracy!  

Excitingly, 6 out of the 7 pigeon pupils were tracked nicely:



When we further evaluate the model accuracy by checking the likelihood of tracked points, we see that the tracking is low confidence when the pigeons close their eyelid (which is of course expected, and can be leveraged to measure blinking 🙄).



But you also might encounter larger problems than small tracking glitches:



To deal with this, you can extract poorly tracked outlier frames, refine them and feed the training data set with them for re-training. Be sure that you set the number of frames to label in the `config.yaml` file of your project folder. The more problems you encounter, the higher the number of frames you might want to label. You should also add the path of the video(s) into the `config.yaml` file, or run the following command to add the videos to your project:

```
deeplabcut.add_new_videos(
    "/pathofproject/config.yaml",
    ["/pathofvideos/pigeon.mp4"],
    copy_videos=False,
    coords=None,
    extract_frames=False
)
```

The `deeplabcut.extract_outlier_frames` function will check for outliers and ask your feedback on whether to extract these outliers frames.

```
deeplabcut.analyze_videos(  
    "/pathofproject/config.yaml",  
    ["/pathofvideos/pigeon.mp4"]  
)  
deeplabcut.extract_outlier_frames(  
    "/pathofproject/config.yaml",  
    ["/pathofvideos/pigeon.mp4"],  
    automatic=True  
)
```

The `deeplabcut.refine_labels` function starts the GUI which allows you to refine the outlier frames manually. You should load the outlier frames directory and corresponding `.h5` file from the previous model. It will ask you to define the `likelihood` threshold: labels under the threshold should be refined at this stage.

After refining, you should combine these data with your previous model's data set and create a new training data set.

```
deeplabcut.refine_labels("/pathofproject/config.yaml",  
deeplabcut.merge_datasets("/pathofproject/config.yaml",  
deeplabcut.create_training_dataset("/pathofproject/co
```

Before starting the training of your model, there is one last step left: editing the `init_weights` parameter in your `pose_cfg.yaml` file. Go to your project and check the latest snapshot (e.g., `snapshot-610000`) of your model in `dlc-models/train` directory. Edit the value of the `init_weights` key in the `pose_cfg.yaml` file and start to re-train your model!

`init_weights:`

`pathofyourproject\dlc-models\iteration-0\DLCFeb31-trainset95shuffle1\train\snapshot-610000`

```
deeplabcut.train_network("/pathofproject/config.yaml",
```





### Hint

Check this video for model refining!

